

APT组织BITTER近期攻击活动相关0DAY漏洞和木马分析

APT组织BITTER近期攻击活动相关0DAY漏洞和木马分析

🕒 2021-03-23 📁 伏影实验室 📌 APT, bitter, oday, 木马, 漏洞

一. 事件回顾

2020年2月24日, 某网络情报公司cyble发布了一篇名为Bitter APT Enhances Its Capabilities With Windows Kernel Zero-day Exploit的文章^[①], 描述了ATP组织Bitter在近期攻击活动中, 使用windows Oday漏洞CVE-2021-1732进行本地提权的行为。该报告同时分析了一个Bitter在近期使用的后门木马。

经分析和整理, 我们发现cyble报告中所述的CVE-2021-1732漏洞, 实际上最早由安恒发现并披露^[②], 并且cyble报告中的漏洞相关的图片内容也直接来自安恒的披露文章。cyble报告中提到的后门木马, 则是由分析组织Shadow Chaser Group发现和披露^[③], 该后门木马的本地程序已被Bitter组织多次使用, 曾出现在2019~2020年的攻击活动中, 分别由绿盟科技^[④]和奇安信^[⑤]及多家其他厂商披露和分析。

目前, CVE-2021-1732漏洞已出现在野利用程序, 利用代码也已在github公布, 但我们尚未发现该漏洞与上述后门木马的直接联系, cyble报告中也未体现。由后门木马的逻辑推断, CVE-2021-1732漏洞可能出现在远程服务器保存的某一攻击组件当中。

二. 漏洞分析

2.1 情况简介

CVE-2021-1732为微软2月月度更新中修复的漏洞, 根据微软官方介绍该漏洞^[⑥]为可利用。2021年3月5日在github上出现对于CVE-2021-1732的公开利用程序, 经验证该可以在未打补丁的系统中实现漏洞利用。

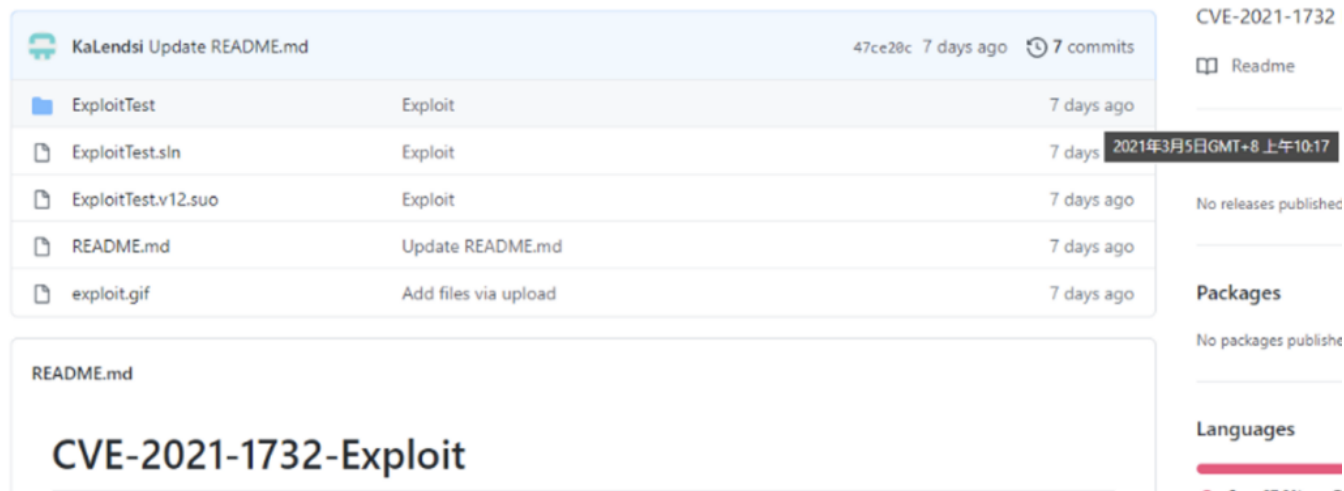


图 1 CVE-2021-1732在Github公开的利用代码

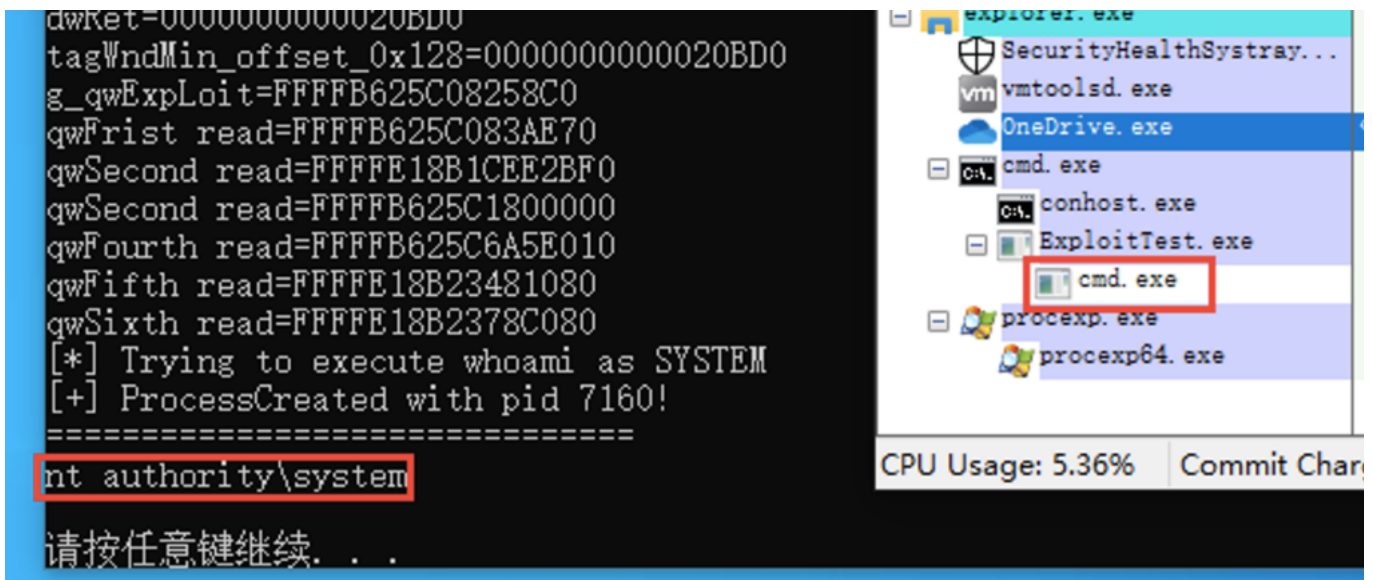


图2CVE-2021-1732的利用效果

2.2 原理分析

CVE-2021-1732是win32kfull.sys中的一个越界写漏洞，成功利用该漏洞可以实现本地提权。漏洞成因是在win32kfull!xxxCreateWindowEx回调xxxClientAllocWindowsClassExtraBytes时产生，通过混淆console窗口和一般窗口，该回调将导致内核对象的越界访问。

该漏洞涉及的主要结构为WNDCLASSEX[⑦]，该结构的定义如下：

```
typedef struct tagWNDCLASSEXA {
    UINT        cbSize;
    UINT        style;
    WNDPROC      lpfnWndProc;
    int         cbClsExtra;
    int         cbWndExtra;
    HINSTANCE    hInstance;
    HICON        hIcon;
    HCURSOR      hCursor;
    HBRUSH       hbrBackground;
    LPCSTR       lpstrMenuName;
    LPCSTR       lpstrClassName;
    HICON        hIconSm;
} WNDCLASSEXA, *PWNDCLASSEXA, *NPWNDCLASSEXA, *LPWNDCLASSEXA;
```

其中cbWndExtra即为本漏洞涉及字段。

当通过win32kfull!xxxCreateWindowEx函数创建一个带扩展内存的窗口时，win32kfull!xxxCreateWindowEx会调用win32kfull!xxxClientAllocWindowClassExtraBytes回调，并在用户态将窗口转换为console窗口。

```
931 | if ( tagWND::RedirectedFieldcbwndExtra<int>::operator!=(u42 + 0xB1, &u260) )
932 | {
933 |     *((u42 + 5) + 296i64) = xxxClientAllocWindowClassExtraBytes(*((u42 + 5) + 200i64));
934 |     u305 = 0i64;
935 |     if ( tagWND::RedirectedFieldpExtraBytes::operator==(unsigned __int64)(u42 + 320, &u305) )
936 |     {
00106652 | xxxCreateWindowEx:915
```

图3win32kfull!xxxCreateWindowEx

```

1 const void *__fastcall xxxClientAllocWindowClassExtraBytes(SIZE_T Length)
2 {
3     SIZE_T v1; // rdi@1
4     int v2; // ebx@2
5     __int64 *v3; // rcx@4
6     const void *v4; // rbx@7
7     __int64 v5; // rax@7
8     const void *result; // rax@7
9     unsigned __int64 v7; // [sp+30h] [bp-38h]@2
10    const void *v8; // [sp+38h] [bp-30h]@7
11    char v9; // [sp+70h] [bp+8h]@2
12    char v10; // [sp+78h] [bp+10h]@2
13    int v11; // [sp+80h] [bp+18h]@1
14    int v12; // [sp+88h] [bp+20h]@3
15
16    v1 = Length;
17    v11 = Length;
18    if ( *gdwInAtomicOperation && *gdwExtraInstrumentations & 1 )
19        KeBugCheckEx(0x160u, *gdwInAtomicOperation, 0i64, 0i64, 0i64);
20    ReleaseAndReacquirePerObjectLocks::ReleaseAndReacquirePerObjectLocks(&v10);
21    LeaveEnterCritProperDisposition::LeaveEnterCritProperDisposition(&v9);
22    EtwTraceBeginCallback(0x7Bi64);
23    v2 = KeUserModeCallback(0x7Bi64, &v11, 4i64, &v7);

```

图4win32kfull!xxxClientAllocWindowClassExtraBytes

此时调用win32kfull!NtUserConsoleControl将改变pExtraBytes的值，pExtraBytes扩展内存的值有两种情况，分别为内存指针或该内存的内核偏移。其中console窗口在pExtraBytes中保存其在堆中的偏移，针对其他类型窗口保存的值为用户态扩展内存的指针。通过回调该窗口被转换为console窗口，创建窗口的函数将改变pExtraBytes的用户态指针。随后当再次调用时，该值被传到内核中，从而引起越界访问。

```

132     pExtraBytesDeskheap = *(v20 + 296) + (*(v15 + 24) + 128i64);
133 }
134 else
135 {
136     LODWORD(v22) = DesktopAlloc(*(v15 + 24), *(v20 + 200), 0i64);
137     pExtraBytesDeskheap = v22;
138     if ( !v22 )
139     {
140         v5 = 0xC0000017;
141 LABEL_33:
142         ThreadUnlock1();
143         return v5;
144     }
145     if ( *(v16 + 0x128i64) )
146     {
147         LODWORD(v23) = PsGetCurrentProcess();
148         v24 = v23;
149         v25 = *(v16 + 200i64);
150         v31 = *(v16 + 296i64);
151         memmove(pExtraBytesDeskheap, v31, v25);
152         if ( !(*(v24 + 780) & 0x40000000) )
153             xxxClientFreeWindowClassExtraBytes(v15, (*(v15 + 40) + 296i64));
154     }
155     *(v16 + 296i64) = pExtraBytesDeskheap - (*(v15 + 24) + 128i64);
156 }
157 if ( pExtraBytesDeskheap )
158 {
159     *pExtraBytesDeskheap = *(v4 + 2);
160     *(pExtraBytesDeskheap + 4) = *(v4 + 3);
161 }
162 *(v16 + 232i64) |= 0x800u;
163 goto LABEL_33;

```

0004447E xxxConsoleControl:160

图 5win32kfull!xxxConsoleControl

针对该漏洞的触发，需要先对函数xxxClientAllocWindowClassExtraBytes进行hook并修改，从而在win32kfull!xxxClientAllocWindowClassExtraBytes回调前调用win32kfull!NtUserConsoleControl进而调用win32kfull!xxxConsoleControl修改pExtraBytes的值。

图 6win32kfull!NtUserConsoleControl

漏洞触发流程图：

图7漏洞触发流程

2.3 调试过程

判断extrabytes的值是否为0，此处poc设置为0x12af：

图 8调试过程1

不为0返回调用win32kfull!xxxClientAllocWindowClassExtraBytes分配内存：

图 9调试过程2

由于回调已经被hook的xxxClientAllocWindowClassExtraBytes：

图10调试过程3

进入hook函数，通过extrabytes位判断是否为目标窗体：

图11调试过程4

调用win32kfull!NtUserConsoleControl:

图12调试过程5

并进而调用win32kfull!xxxConsoleControl:

图13调试过程6

判断标志位，并设置poi(hConsoleWnd+0x28)+0x128为pExtraBytesDeskheap的偏移， poi(hConsoleWnd+0x28)+E8标志位为偏移寻址:

图14调试过程7

图15调试过程8

之后继续手动调用win32kfull! NtCallbackReturn:

图16调试过程9

将申请的pExtraBytesDeskheap的地址改为指定的地址，此处为0xffffffff00:

图17调试过程10

返回win32kfull!xxxCreateWindowEx得到返回值，已经被修改:

图 18调试过程11

随后继续调用win32kfull!xxxClientAllocWindowClassExtraBytes，此时窗口的pExtrabyte已经被设置成为了攻击者指定的值:

图19调试过程12

图 20调试过程13

检查标志位，计算偏移，并使用得到的地址指向的作为位返回值，这里由于指定的地址的值不存在最终造成越界访问：

图 21调试过程14

图22调试过程15

图23调试过程16

图24调试过程17

2.4 利用分析

文件md5值为AC8A521A56ED5F4EF2004D77668C14D0，IDA加载显示的符号文件路径如下：
C:\Users\Win10\source\repos\KSP_EPL\x64\Release\ConsoleApplication13.pdb

图25ExP程序pdb路径

程序主要流程的入口函数为sub_140006A30()，恶意行为在sub_140002570()中：

图 26主要流程入口

遍历进程查找是否有卡巴斯基：

图27遍历杀软进程

判断当前环境是否为x64:

图28x64环境判断

获取RtlGetNtVersionNumbers、NtUserConsoleControl、NtCallbackReturn函数的地址:

图29动态获取api

调用RtlGetNtVersionNumbers判断版本号是否大于16353（1709）和18204（1903），如果满足版本需求进入漏洞利用函数sub_140002080()：

图30版本号判断

首先获取HmValidateHandle函数地址，并对User32!_xxxClientAllocWindowClassExtraBytes函数进行hook：

图 31User32!_xxxClientAllocWindowClassExtraBytes函数hook

随后注册两个窗体类，一个正常的，一个魔术类用于创建触发漏洞窗体：

图32注册窗体类

利用过程中，首先创建10个正常窗体，调用HValidateHandle获取每个窗体的tagWND地址，随后删除后8个window只保留0号和1号。

图33获取tagWND地址

图34删除窗体

如果当前程序是64位，输入window 0的handle并修改WndExtra字段偏移。接着泄露window 0的内核tagWND地址。

随后创建magicClass窗体，该窗体cbWndExtra为注册时的指定值，在创建过程中将会调用win32kfull!xxxClientAllocWindowClassExtraBytes回调函数，进入之前的hook函数中。

图35创建窗体，触发回调

在hook函数中，首先检查cbWndExtra是否为magic字，并判断是否为64位程序，当都通过后调用NtUserConsolControl传入magic window的handle，改变其WndExtra为偏设置相关标志位。接着调用NtCallbackReturn并传入window 0的内核tagWND。当返回内核态后，magic window的WndExtra偏移将会修改为window 0的内核tagWND偏移后实现对其的读写操作。

图36hook函数逻辑

magic window创建后，程序将通过设置magic window的WndExtra字段修改window 0 的内核tagWND。接着调用SetWindowLongW测试测试权限。

测试通过后，调用SetWindowLongW修改window0的cbWndExtra为0xffffffff，使其有权限越界读写。接着修改window 1的类型为WS_CHILD，从而替换window 1的spmer伪造的spmenu。

图37改造window0与window1

任意地址读权限通过函数GetMenuBarInfo获得，该程序通过使用tagMenuBarInfo.rcBar.left和tagMenuBarInfo.rcBar.top读取4字节。

图38实现任意地址读

任意地址写通过window 0 和window 1以及Set WindowLongPtrA配合使用获取。

图39实现任意地址写

完成获取读写权限后，程序从原始的spmenu中获取内核地址，接着搜索当前程序的EPROCESS结构。

最终该程序遍历ActiveProcessLinks表获取SYSTEM进程的EPROCESS和当前进程EPROCESS的Token，进行替换实现提权。

图 40替换Token

之后恢复window 0、window 1和magic window的参数完成所有操作。

三. 木马分析

3.1 初始载荷：7b64a739836c6b436c179eac37c446fee5ba5abc6c96206cf8e454744a0cd5f2

该文件是WinRAR自解压文件，其运行后主要行为是：

1. 释放并打开诱饵文档CICP Z9 Letter dated December 2020.docx
2. 释放并运行恶意可执行文件dlhosts.exe

3.2 诱饵文档：CICP Z9 Letter dated December 2020.docx(a36b066fd9aaab9cc6619873dfeebef50240844d31b0b08dda13085becb9286d)

该文档是用于伪装的诱饵文件，打开后显示无意义乱码，根据字符排布可知，该乱码信息完全由人工输入：

图41诱饵文档内容

3.3 主要载荷：dlhosts.exe(26b3c9a5077232c1bbb5c5b4fc5513e3e0b54a735c32ae90a6d6c1e1d7e4cc0f)

该程序是一个简单的下载者木马，可以用于执行从CnC处下载的攻击组件。该下载者木马是Bitter组织的惯用木马，至少在2019年就已经出现。

3.3.1 行为

该木马在启动后首先对字符串进行解密，解密逻辑为逐字节减0xD：

图42字符串解密逻辑

创建信号量7t56yr54r，保证进程的唯一性：

图43创建信号量

随后，木马收集宿主机信息，用于构建上线通信请求。

3.3.2 通信

该木马与硬编码CnC地址82.221.136.27通信，发送信息，下载CnC处的攻击载荷并执行。

该木马构建的首个HTTP请求中包含了收集到的宿主机信息，各参数字符及内容对应如下：

参数名	参数内容
a	主机名
b	计算机名
c	操作系统版本
d	当前账户、MachineGuid
e	固定标记"efgh"

表格1主要载荷HTTP请求参数信息对应

图 44主要载荷通信流量

CnC服务器对该请求的响应分为两种情况。

情况一：

回复包正文中包括“`Yes file`”字符串。

此时木马程序会寻找回复包正文里使用“`[`”和“`]`”包裹的文件名关键字，随后发送HTTP GET请求，将`http://82.221.136.27/RguhsT/RguhsT/`目录下的文件名关键字对应的文件至本地，作为exe文件运行。

情况二：

回复包正文中包括“`No file`”字符串。

此时木马会放弃该次HTTP通信，重复与CnC连接并发送初始HTTP请求的过程，直到进入情况一为止。

3.4 后续载荷：持久化组件(b2d7336f382a22d5fb6899fc2bd87c7cd401451ecd6af8ccb9ea7dbbe62fc1b7)

该文件是dlhost.exe曾经下载并使用过的攻击组件，用于将名为audiodq的程序设置为自启动项，其字符串解密逻辑与dlhost.exe木马程序相同：

图 45持久化组件主要功能代码

audiodq是一个简单的下载器程序，曾在Bitter组织早期的攻击活动中投入使用过，负责根据C2下发的任务，下载不同的模块到受感染机器中。我们尚未找到对应在本次的活动中的audiodq关联程序本体。

3.5 后续载荷：间谍木马(d957239ba4d314e47de9748e77a229f4f969f55b3fcf54a096e7971c7f1bab7d)

该文件是dlhost.exe曾经下载并使用过的攻击组件，是一种间谍木马，用于收集本机信息并上传收集到的信息，其字符串解密逻辑与其他木马程序相似，为逐字节加减运算。

该木马会收集宿主机上各物理磁盘和可移动介质上的txt、ppt、pptx、pdf、doc、docx、xls、xlsx、zip、z7、rtf.txt、apk、jpg、jpeg后缀名类型的文件的路径和内容等信息并将这些信息分别发送给硬编码CnC地址72.11.134.216处。

文件路径信息相关示例流量如下，其url参数部分携带了计算机名、MachineGuid、时间戳等内容：

图 46间谍木马通信流量A

文件内容信息相关示例流量如下：

图 47间谍木马通信流量B

3.6 后续载荷：RAT木马(78b16177d8c5b2e06622688a9196ce7452ca1b25a350daae8c4f12c2e415065c)

该文件是dlhost.exe曾经下载并使用过的攻击组件，自称为Splinter，是使用C#编写的RAT木马程序。该程序同样曾在早期的Bitter攻击活动中出现过，伏影实验室曾对捕获该程序进行了披露和分析（<http://blog.nsfocus.net/splinters-new-apt-attack-tool-dialysis/>）。新版程序中，Splinter疑似经历了一些版本迭代，优化了代码和功能。

该实例连接的CnC地址为pichostfrm.net:58370：

图48RAT木马CnC地址储存区域

该实例协议结构与功能对应如下表：

Packet Len(2 bytes)	Meanings	Plain CmdCode(1byte)	Cyphered CmdCode(1 byte)	Params
According to the packet	Delete File	2	0xF4	FileLocation
According to the packet	FileMgr get drives	18	0xE4	NULL
According to the packet	FileMgr get Folders	19	0xE5	DirLocation
According to the packet	FileMgr Create File	20	0xE2	FileLocation/FileName
According to the packet	FileMgr Copy File	21	0xE3	FileLocation/New FileLocation
According to the packet	FileTransfer Begin	38	0xD0	File Id/File Name/File Destination/File Size/File Type
According to the packet	FileTransfer Data	39	0xD1	File Id/Length/Index/Total File Length/File Bytes
According to the packet	FileTransfer Complete	40	0xDE	File Id
According to the packet	FileTransfer for downloading start	41	0xDF	File Id/File Name/File Destination/File Size/File Type
According to the packet	Get Command	48	0xC6	command
According to the packet	Start Command Prompt	49	0xC7	NULL
According to the packet	Stop Command Prompt	50	0xC4	NULL
According to the packet	Connection Status	51	0xC5	NULL

表格2RAT木马协议结构与功能对应表

该 Splinter流量使用单字节异或加密，异或键为0xCA。

相比早期版本，该Splinter示例精简掉了进程管理、剪贴板管理、获取CPU信息等功能，推测做出这些改变的原因为其剥离至其他组件实现。

四. 组织关联

该报告中描述的漏洞CVE-2021-1732，最早出现在由安恒披露的某Bitter组织攻击组件[⑧]中，用于进行本地提权。

该事件中的主要载荷dlhost.exe，曾被用于Bitter（蔓灵花）组织在2020年底的攻击行动[⑨]中，MD5值一致（25a16b0fca9acd71450e02a341064c8d）。

因此本次攻击中的有效载荷实际上是使用winrar重新包装的老旧木马，推测其后续攻击过程与已有报告中的描述一致。

五. IoC

CVE-2021-1732在野利用	914b6125f6e39168805fdf57be61cf20dd11acd708d7db7fa37ff75bf1abfc29
初始载荷	7b64a739836c6b436c179eac37c446fee5ba5abc6c96206cf8e454744a0cd5f2
诱饵文档	a36b066fd9aaab9cc6619873dfeebef50240844d31b0b08dda13085becb9286d
主要载荷	26b3c9a5077232c1bbb5c5b4fc5513e3e0b54a735c32ae90a6d6c1e1d7e4cc0f
后续载荷-持久化组件	b2d7336f382a22d5fb6899fc2bd87c7cd401451ecd6af8ccb9ea7dbbe62fc1b7
后续载荷-间谍木马	d957239ba4d314e47de9748e77a229f4f969f55b3fcf54a096e7971c7f1bab7d
后续载荷-RAT木马	78b16177d8c5b2e06622688a9196ce7452ca1b25a350daae8c4f12c2e415065c
主要载荷CnC IP	82.221.136.27
主要载荷CnC url	hxxp://82.221.136.27///RguhsT/accept.php
间谍木马CnC IP	72.11.134.216
间谍木马CnC url	hxxp://72.11.134.216/autolan.php
RAT木马CnC domain	pichostfrm.net:58370

关于伏影实验室

研究目标包括Botnet、APT高级威胁，DDoS对抗，WEB对抗，流行服务系统脆弱利用威胁、身份认证威胁，数字资产威胁，黑色产业威胁及新兴威胁。通过掌控现网威胁别风险，缓解威胁伤害，为威胁对抗提供决策支撑。

关于天机实验室

专注于漏洞挖掘与利用技术研究。研究方向主要包括漏洞挖掘技术研究、漏洞分析技术研究、漏洞利用技术研究、安全防御机制及对抗技术研究等。研究目标涵盖主流操作系统、流行的应用系统及软件、重要的基础组件库以及新兴的技术方向。

- [①] <https://cybleinc.com/2021/02/24/bitter-apt-enhances-its-capability-with-windows-kernel-zero-day-exploit/>
- [②] <https://ti.dbappsecurity.com.cn/blog/index.php/2021/02/10/windows-kernel-zero-day-exploit-is-used-by-bitter-apt-in-targeted-attack-cn/>
- [③] <https://twitter.com/ShadowChasing1/status/1362686004725866502>
- [④] <http://blog.nsfocus.net/splinters-new-apt-attack-tool-dialysis/>
- [⑤] <https://ti.qianxin.com/blog/articles/Blocking-APT:-Qianxin's-QOWL-Engine-Defeats-Bitter's-Targeted-Attack-on-Domestic-Government-and-Enterprises/>
- [⑥] <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2021-1732>
- [⑦] <https://docs.microsoft.com/en-us/windows/win32/api/winuser/ns-winuser-wndclasexa>
- [⑧] <https://ti.dbappsecurity.com.cn/blog/index.php/2021/02/10/windows-kernel-zero-day-exploit-is-used-by-bitter-apt-in-targeted-attack-cn/>
- [⑨] <https://ti.qianxin.com/blog/articles/Blocking-APT:-Qianxin's-QOWL-Engine-Defeats-Bitter's-Targeted-Attack-on-Domestic-Government-and-Enterprises/>

Spread the word. Share this post!

MEET THE AUTHOR

伏影实验室

伏影实验室专注于安全威胁监测与对抗技术研究。

研究目标包括Botnet、APT高级威胁，DDoS对抗，WEB对抗，流行服务系统脆弱利用威胁、身份认证威胁，数字资产威胁，黑色产业威胁及新兴威胁。通过掌控现网威胁来识别风险，缓解威胁伤害，为威胁对抗提供决策支撑。

LEAVE COMMENT

要发表评论，您必须先[登录](#)。
